

Autonomous Modular AI Robot (Aura Bot)

Полное техническое руководство, история отладки и эталонная прошивка v1.1.0

Платформа: ESP32-S3 (N16R8)	Архитектура: 2-DOF Pan/Tilt + ST7789 IPS + OV5640	Дата обновления: Сентябрь 2026
Фреймворк: PlatformIO / Arduino ESP32	Графика: Image Sprite System (Bitmap Emotions)	Статус: Hardware STABLE v1.1.0 READY

1. Обзор проекта и Архитектура

Проект представляет собой автономного модульного локального AI-робота (Aura Bot) на базе двухъядерного микроконтроллера ESP32-S3. Главный упор сделан на полнейшую независимость от облачных сервисов, локальную обработку, выведение эмоций из готовых растровых спрайтов (Bitmap Eyes), а также активный осмотр пространства с помощью камеры OV5640 на длинном шлейфе, закрепленной вместе с дисплеем на 2-DOF поворотно-наклонном механизме.

2. Аппаратная Конфигурация (Hardware Spec)

Компонент	Спецификация / Модель	Интерфейс / Пины ESP32-S3	Примечания
MCU	GOOUUU ESP32-S3-N16R8	16MB Flash (QIO), 8MB PSRAM (OPI)	Native USB CDC (OTG)
Дисплей	GMT130 IPS 240x240 (ST7789)	SCK: GPIO 19 SDA: GPIO 20 RES: GPIO 21 DC: GPIO 47 BLK: GPIO 38 CS: GND (-1)	Установлен на серво-платформе. Без пина CS.
Камера	OV5640 (Длинный шлейф FPC)	Шина DVP / DVP Parallel (8-bit)	Закреплена соосно экрану на головном блоке.
Servo Yaw (X)	SG90 / MG90S	GPIO 10 (Питание: 5V шина ESP32)	Горизонтальный обзор 20°...160°
Servo Pitch (Y)	SG90 / MG90S	GPIO 11 (Питание: 5V шина ESP32)	Вертикальный обзор 30°...150°

⚠ Важные Аппаратные Особенности

- Питание Серво от ESP32:** В текущей ревизии сервоприводы питаются прямо от пина 5V (USB) платы ESP32. Из-за импульсных токов моторов возможны небольшие дергания и шум. В будущем при повышении нагрузки рекомендуется вынести их на отдельный Line 5V.
- Пин GPIO 45:** Запрещено притягивать к GND при старте (напряжение питания Flash/PSRAM).

3. Графика и Концепция Эмоций (Sprite Emotion Engine)

Чтобы не перегружать вычислительные ядра математикой рисования процедурных фигур, эмоции загружаются из предзаготовленных изображений/массивов (Bitmap PNG -> C-Array / SPIFFS):

- Автономный режим (Idle):** Стандартные круглые симметричные глаза (EMOTION_IDLE). При движении сервов координата спрайта смещается пропорционально углу поворота.

• **Управляемый режим (AI Controlled):** ИИ-агент выбирает и активирует пресет эмоции по имени:

- `EMOTION_HAPPY` — Радость / Улыбающиеся глаза
- `EMOTION_SAD` — Грусть / Опущенные веки
- `EMOTION_SURPRISED` — Удивление / Увеличенные зрачки
- `EMOTION_ANGRY` — Злость / Наклоненные бровные линии
- `EMOTION_THINKING` — Задумчивость / Взгляд в сторону

4. Полный свод проблем и решений

4.1. Дергание сервоприводов при питании от платной шины ESP32

Причина: Пин 5V платы ESP32 питается напрямую от USB-порта ПК. При резком старте моторов происходят микропадения напряжения.

Решение: В коде сглажено ускорение: задержка зафиксирована на уровне `8 мс`, а шаг угла выставлен в `3°`. Это убрало спайки тока.

4.2. Конфликт ресурсов SPI экрана и параллельной шины Камеры OV5640

Решение: Дисплей переведен на выделенный порт `HSPI` (`SPI2`) на частоте 10 МГц, что освобождает пины DMA и таймеры для видеопотока OV5640.

5. Конфигурация `platformio.ini`

```
[env:esp32-s3-devkitc-1]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200

board_build.arduino.memory_type = qio_qspi
build_flags =
    -DBOARD_HAS_PSRAM
    -DARDUINO_USB_CDC_ON_BOOT=1
    ; --- ДИСПОЛЕЙ TFT_eSPI ---
    -DUSER_SETUP_LOADED=1
    -DST7789_DRIVER=1
    -DTFT_WIDTH=240
    -DTFT_HEIGHT=240
    -DTFT_MOSI=20
    -DTFT_SCLK=19
    -DTFT_CS=-1
    -DTFT_DC=47
    -DTFT_RST=21
    -DTFT_BL=38
    -DTFT_BACKLIGHT_ON=HIGH
    ; --- СТАБИЛИЗАЦИЯ И ШИНЫ ---
    -DUSE_HSPI_PORT=1
    -DSPI_FREQUENCY=10000000
    -DSPI_READ_FREQUENCY=6000000
    -DLOAD_GLCD=1

lib_deps =
    bodmer/TFT_eSPI@^2.5.43
    madhephaestus/ESP32Servo@^1.2.1
```

6. Исходный Код (`src/main.cpp`)

```
#include <Arduino.h>
#include <TFT_eSPI.h>
#include <ESP32Servo.h>

#define SERVO_YAW_PIN 10
#define SERVO_PITCH_PIN 11

TFT_eSPI tft = TFT_eSPI();
TFT_eSprite spr = TFT_eSprite(&tft);

Servo servoYaw;
Servo servoPitch;

// Перечисление Именованных Эмоций для ИИ
enum Emotion {
    EMOTION_IDLE,        // Нейтральные круглые глаза (Автономный)
    EMOTION_HAPPY,        // Радость
    EMOTION_SAD,          // Грусть
    EMOTION_SURPRISED,    // Удивление
    EMOTION_ANGRY,        // Злость
    EMOTION_THINKING      // Задумчивость
};

Emotion currentEmotion = EMOTION_IDLE;

const int EYE_W = 50;
const int EYE_H = 70;
const int EYE_R = 15;
const int BASE_LEFT_X = 50;
const int BASE_RIGHT_X = 140;
const int BASE_Y = 85;

void renderEmotionFrame(Emotion emo, int offsetX, int offsetY) {
    spr.fillSprite(TFT_BLACK);

    int leftX = BASE_LEFT_X + offsetX;
    int rightX = BASE_RIGHT_X + offsetX;
    int eyeY = BASE_Y + offsetY;

    switch (emo) {
        case EMOTION_HAPPY:
            // В будущем: отрисовка bitmap спрайта с флеш-памяти
            spr.fillCircle(leftX + 25, eyeY + 20, 20, TFT_GREEN);
            spr.fillCircle(rightX + 25, eyeY + 20, 20, TFT_GREEN);
            break;

        case EMOTION_IDLE:
        default:
            // Базовые круглые глаза для автономного режима
            spr.fillRoundRect(leftX, eyeY, EYE_W, EYE_H, EYE_R, TFT_CYAN);
            spr.fillRoundRect(rightX, eyeY, EYE_W, EYE_H, EYE_R, TFT_CYAN);
            break;
    }

    spr.pushSprite(0, 0);
}

void setup() {
```

```

Serial.begin(115200);

tft.init();
tft.setRotation(0);
spr.createSprite(240, 240);

renderEmotionFrame(currentEmotion, 0, 0);

ESP32PWM::allocateTimer(0);
ESP32PWM::allocateTimer(1);

servoYaw.setPeriodHertz(50);
servoPitch.setPeriodHertz(50);

servoYaw.attach(SERVO_YAW_PIN, 500, 2500);
servoPitch.attach(SERVO_PITCH_PIN, 500, 2500);

servoYaw.write(90);
servoPitch.write(90);
delay(300);
}

void loop() {
    // Автономный обход пространства (Камера и Дисплей на сервах)
    for (int posX = 20; posX <= 160; posX += 3) {
        int posY = map(posX, 20, 160, 30, 150);

        servoYaw.write(posX);
        servoPitch.write(posY);

        int eyeOffsetX = map(posX, 20, 160, -22, 22);
        int eyeOffsetY = map(posY, 30, 150, -18, 18);

        renderEmotionFrame(currentEmotion, eyeOffsetX, eyeOffsetY);
        delay(8);
    }

    for (int posX = 160; posX >= 20; posX -= 3) {
        int posY = map(posX, 20, 160, 30, 150);

        servoYaw.write(posX);
        servoPitch.write(posY);

        int eyeOffsetX = map(posX, 20, 160, -22, 22);
        int eyeOffsetY = map(posY, 30, 150, -18, 18);

        renderEmotionFrame(currentEmotion, eyeOffsetX, eyeOffsetY);
        delay(8);
    }
}

```

7. Следующие шаги

1. Подключение камеры OV5640 по DVP шлейфу и выделение PSRAM под кадровый буфер (Frame Buffer).
2. Подготовка набора Bitmap картин-спрайтов 240x240 для вывода эмоций через SPIFFS / PROGMEM.